

Torneo de Programación UCM

Cuadernillo de problemas



Universidad Católica del Maule

Ingeniería Civil Informática

Escrito y creado por

Gustavo Gallegos Harnisch

2026

Problemas A–P

Reglas de competencia

Formato local inspirado en ICPC

Estas reglas adaptan al torneo UCM el formato de las [Reglas Regionales ICPC 2026/27](#). Ante cualquier situación no prevista, la decisión del director de competencia y del equipo de jueces será final.

¡EXPULSIÓN INMEDIATA!

EXPULSIÓN INMEDIATA

Si un equipo es sorprendido consultando **información en Internet** o utilizando **inteligencia artificial generativa**, será **expulsado inmediatamente de la competencia**.

Esta prohibición incluye, entre otros, buscadores, páginas de documentación en línea, foros, repositorios remotos, servicios de mensajería, ChatGPT, Gemini, Claude, GitHub Copilot y cualquier herramienta que genere, complete, traduzca, explique o depure código mediante IA. La sanción se aplica al **equipo completo**, independientemente de cuál integrante haya realizado la acción.

1. Equipos y puesto de competencia

- Cada equipo estará formado por tres integrantes y utilizará una sola estación de trabajo.
- Los integrantes pueden conversar y compartir ideas únicamente dentro de su propio equipo.
- Durante la competencia solo podrán comunicarse con jueces o personal autorizado, principalmente mediante solicitudes de aclaración.
- No se permite interferir con otros equipos, modificar el entorno de competencia ni afectar deliberada o accidentalmente la infraestructura.

2. Envíos, aclaraciones y decisiones

- Una solución se considera resuelta cuando recibe veredicto **Accepted** por parte del juez.
- Si un enunciado parece ambiguo o incorrecto, el equipo debe enviar una solicitud de aclaración. Las respuestas relevantes para todos serán comunicadas de forma general.
- Los jueces son responsables de aceptar o rechazar los envíos. El director podrá resolver situaciones imprevistas y ajustar condiciones si existe una dificultad técnica general.

3. Clasificación y penalización

Los equipos se ordenan según el sistema clásico ICPC:

1. mayor cantidad de problemas resueltos;
2. menor tiempo total de penalización;
3. si aún fuese necesario, menor tiempo del último envío aceptado.

Para cada problema resuelto, el tiempo corresponde a los minutos transcurridos desde el inicio hasta el primer envío aceptado, más **20 minutos por cada envío rechazado anterior** de ese mismo problema. Los errores de compilación no añaden penalización. Un problema no resuelto no aporta tiempo.

4. Material permitido

- Se permite utilizar este cuadernillo, hojas para borrador, lápices y los programas instalados y autorizados por la organización.
- Se permiten las bibliotecas estándar del lenguaje y el material local que la organización haya dejado disponible.
- No se permiten teléfonos, relojes inteligentes, dispositivos externos, mensajería, correo electrónico, repositorios remotos ni comunicación con personas ajenas al equipo.
- Todo el código enviado debe ser producido por los integrantes del equipo durante la competencia, salvo plantillas expresamente autorizadas por la organización.

5. Conducta y juego limpio

- Se exige respeto hacia participantes, jueces, voluntarios y personal técnico.
- Cualquier intento de acceder a cuentas ajenas, sabotear equipos, alterar datos del juez, compartir soluciones o recibir ayuda externa puede causar expulsión.
- Los problemas técnicos deben informarse de inmediato. El personal de soporte puede explicar errores del sistema, pero no ayudar a resolver los problemas.
- Participar implica aceptar estas reglas y las decisiones de la organización.

Ayuda de compilación

C++17 y Python 3

Esta referencia está pensada para trabajar localmente durante la competencia. Los nombres de archivo utilizados son ejemplos y pueden cambiarse.

C++17 con GNU g++

Plantilla recomendada

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // Solución

    return 0;
}
```

El encabezado `<bits/stdc++.h>` carga casi toda la biblioteca estándar de C++. Es una extensión de GNU, habitual en programación competitiva y disponible con g++; no forma parte del estándar ISO C++ ni es necesariamente portable a otros compiladores.

Compilar y ejecutar

Guarda el programa como `main.cpp` y ejecuta:

```
g++ -std=gnu++17 -O2 -pipe -Wall -Wextra -o main main.cpp
./main < input.txt > output.txt
```

Para una prueba interactiva simple, ejecuta `./main` y escribe la entrada manualmente. Los mensajes de depuración deben enviarse a `cerr`, nunca a la salida requerida por el problema.

Bibliotecas y utilidades frecuentes de C++

Elemento	Uso habitual
<code>vector</code> , <code>array</code> , <code>string</code>	Arreglos dinámicos, arreglos fijos y texto.
<code>deque</code> , <code>queue</code> , <code>stack</code>	Colas dobles, colas FIFO y pilas LIFO.
<code>set</code> , <code>multiset</code> , <code>map</code>	Colecciones ordenadas en $O(\log n)$.
<code>unordered_set</code> , <code>unordered_map</code>	Tablas hash con acceso promedio $O(1)$.
<code>priority_queue</code>	Heaps de máximos o mínimos.
<code>pair</code> , <code>tuple</code>	Agrupar dos o más valores.
<code>sort</code> , <code>stable_sort</code> , <code>reverse</code>	Ordenar y reorganizar rangos.
<code>lower_bound</code> , <code>upper_bound</code>	Búsqueda binaria sobre rangos ordenados.
<code>min</code> , <code>max</code> , <code>min_element</code> , <code>max_element</code>	Mínimos y máximos.
<code>accumulate</code> , <code>iota</code> , <code>gcd</code> , <code>lcm</code>	Sumas, secuencias y aritmética básica.
<code>bitset</code>	Operaciones compactas sobre bits.

Usa `long long` para enteros de hasta aproximadamente $9 \cdot 10^{18}$. Cuando una multiplicación pueda desbordar antes de ser almacenada, convierte un operando a `__int128`.

Comprobación local

Comparar la salida

Si guardaste la respuesta esperada en `expected.txt`, puedes comparar:

```
diff -u expected.txt output.txt
```

Revisa espacios, saltos de línea, mayúsculas y el formato exacto solicitado. Elimina mensajes de depuración antes de enviar.

Python 3

Plantilla recomendada

```
import sys

input = sys.stdin.readline

def main():
    n = int(input())
    values = list(map(int, input().split()))
    print(n, len(values))

if __name__ == "__main__":
    main()
```

Ejecutar

Guarda el programa como `main.py` y ejecuta:

```
python3 main.py < input.txt > output.txt
```

Para entradas muy grandes puede resultar más rápido leer todos los datos:

```
data = sys.stdin.buffer.read().split()
numbers = list(map(int, data))
```

Módulos útiles de Python

Módulo	Elementos frecuentes
<code>math</code>	<code>gcd</code> , <code>lcm</code> , <code>isqrt</code> , <code>ceil</code> , <code>floor</code> , <code>inf</code> .
<code>collections</code>	<code>deque</code> , <code>Counter</code> , <code>defaultdict</code> .
<code>heapq</code>	Heap de mínimos y colas de prioridad.
<code>bisect</code>	Búsqueda e inserción binaria ordenada.
<code>itertools</code>	<code>permutations</code> , <code>combinations</code> , <code>product</code> , <code>accumulate</code> .
<code>functools</code>	<code>lru_cache</code> , <code>cache</code> , <code>reduce</code> .
<code>operator</code>	Funciones de operadores para ordenamiento y reducción.

Python utiliza enteros de precisión arbitraria. Aun así, los valores enormes pueden hacer más lentos los cálculos. Evita la recursión profunda; si resulta imprescindible, configura `sys.setrecursionlimit(...)` con cautela.

Índice de problemas

Problema	Título	Página
A	Cyberpunk 2077	7
B	Stardew Valley	9
C	Kerbal Space Program	12
D	Papers, Please	14
E	Factorio	16
F	Pokémon	18
G	Guitar Hero 3	20
H	Moving Out	22
I	Balatro	24
J	Minecraft	26
K	FIFA	28
L	Terraria	30
M	Oxygen Not Included	32
N	StarCraft	34
O	Among Us	36
P	Portal 2	38

A. Cyberpunk 2077

Tiempo: 1 s**Memoria:** 512 MB**Entrada:** entrada estándar**Salida:** salida estándar

En lo profundo de Night City, V consigue acceder a uno de los antiguos servidores de Arasaka. Entre los archivos abandonados aparece algo particularmente interesante: dos versiones diferentes de una misma clave de acceso utilizada por uno de sus sistemas internos.

La primera versión corresponde a una copia antigua almacenada antes de una actualización del sistema, mientras que la segunda es la versión que permanece actualmente en los servidores de Arasaka. Después de años de modificaciones, corrupción de datos y cambios en el software, algunas letras pudieron haber sido añadidas, eliminadas o reemplazadas.

Para analizar cuánto ha cambiado la clave original, V necesita determinar la menor cantidad de modificaciones necesarias para transformar una versión en la otra.

La distancia de edición entre dos cadenas es el número mínimo de operaciones necesarias para transformar una cadena en la otra.

Las operaciones permitidas son:

- Añadir un carácter a la cadena.
- Eliminar un carácter de la cadena.
- Reemplazar un carácter de la cadena por otro.

Por ejemplo, la distancia de edición entre las cadenas LOVE y MOVIE es 2. La letra L puede reemplazarse por M para obtener MOVE, y luego se puede añadir la letra I para obtener MOVIE. Por lo tanto, solo se necesitan 2 operaciones para transformar una cadena en la otra.

Antes de continuar con la infiltración, V necesita conocer exactamente qué tan diferente es la versión actual de la clave respecto de la copia encontrada en el servidor.

Tu tarea es ayudar a V a calcular la distancia mínima de edición entre las dos versiones del código.

Entrada

La entrada contiene dos líneas.

La primera línea contiene una cadena de n caracteres entre A–Z, que representa la primera versión del código.

La segunda línea contiene una cadena de m caracteres entre A–Z, que representa la segunda versión del código.

Restricciones:

- $1 \leq n, m \leq 5000$

Salida

Imprima un único entero — la distancia mínima de edición entre las dos cadenas.

Ejemplo

Entrada

```
LOVE  
MOVIE
```

Salida

```
2
```

Nota

En el primer ejemplo, se realizan las siguientes operaciones:

1. Reemplazar la letra L por M: LOVE $\rightarrow$ MOVE.
2. Añadir la letra I: MOVE $\rightarrow$ MOVIE.

Por lo tanto, la distancia mínima de edición es 2.

B. Stardew Valley

Tiempo: 2 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

Después de una larga jornada recorriendo los alrededores de Pueblo Pelicano, un granjero de Stardew Valley ha decidido ampliar sus terrenos de cultivo. Como suele ocurrir en el valle, los límites de algunas parcelas no están del todo claros, pero eso no parece preocupar demasiado al granjero; después de todo, organizar terrenos y decidir dónde cultivar forma parte de la vida cotidiana en la granja.

Entre las parcelas disponibles existe un terreno rectangular del cual no se conocen exactamente sus dimensiones. Sin embargo, el granjero sabe con certeza que el área del rectángulo es S , que las longitudes de sus lados son enteras y que su esquina inferior izquierda se encuentra en el punto $(0, 0)$.

Antes de comenzar una nueva temporada de cultivo, el granjero quiere estudiar qué partes de una determinada zona podrían pertenecer realmente al terreno. Para ello, considera un rectángulo de lados x e y , cuya esquina inferior izquierda también se encuentra en el punto $(0, 0)$, y quiere saber cuántas de sus celdas pueden estar ubicadas dentro del terreno agrícola.

Se considera que una celda puede formar parte del terreno si existe algún rectángulo que satisfaga todas las restricciones conocidas sobre la parcela y que contenga dicha celda. Ayuda al granjero de Pueblo Pelicano a responder rápidamente sus consultas antes de que termine el día. Debes responder q consultas de este tipo.

Entrada

Cada prueba contiene múltiples casos de prueba. La primera línea contiene el número de casos de prueba t ($1 \leq t \leq 10\,000$). A continuación se describe cada caso de prueba.

La primera línea de cada caso de prueba contiene dos enteros S y q — el área del terreno agrícola y el número de consultas ($1 \leq S \leq 10^{14}$; $1 \leq q \leq 3 \cdot 10^5$).

A continuación siguen q líneas, cada una de las cuales contiene dos enteros x, y — la siguiente consulta ($1 \leq x, y \leq S$).

Se garantiza que la suma de q sobre todos los casos de prueba no excede $3 \cdot 10^5$.

Se garantiza que la suma de $\sqrt{S}$ sobre todos los casos de prueba no excede 10^7 .

Salida

Para cada consulta, imprime un único entero en una línea separada — la respuesta a la consulta.

Ejemplo

Entrada

```
3
6 4
2 3
4 5
6 6
1 1
5 2
2 2
3 4
8 2
3 1
5 6
```

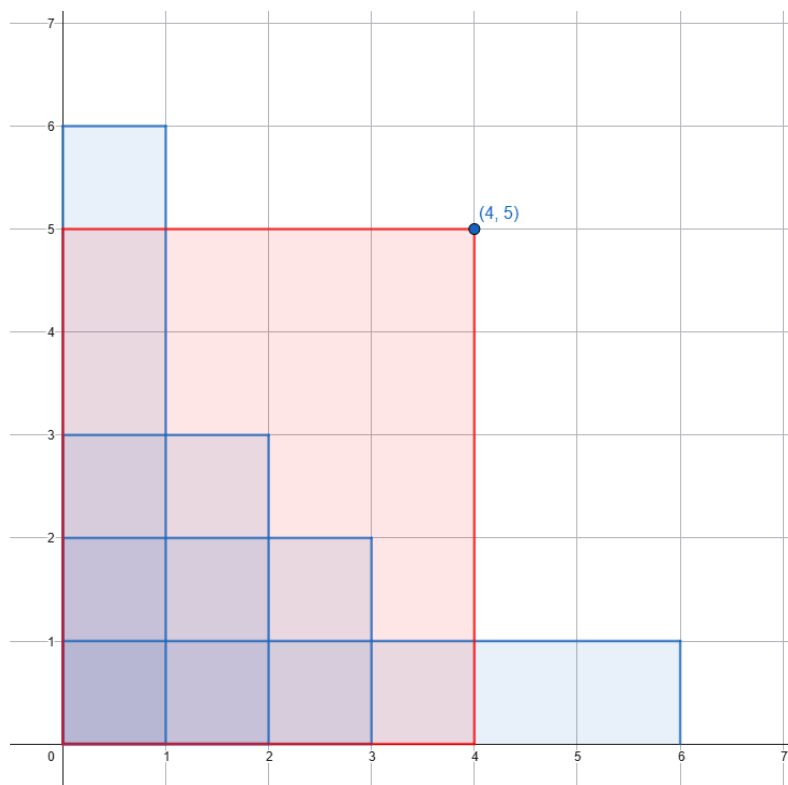
Salida

```
6
11
14
1
3
6
3
15
```

Nota

En el primer caso de prueba, todas las celdas del rectángulo $(2, 3)$ se cuentan en la primera consulta, ya que dicho rectángulo puede corresponder exactamente a uno de los posibles terrenos de la granja.

La segunda consulta del primer caso de prueba se ilustra en la figura. La respuesta a la consulta corresponde al número de celdas que se encuentran en la intersección de las regiones azul y roja.



4 rectángulos azules — posibles configuraciones del terreno de la granja. Rojo — el rectángulo de la consulta
 $(x, y) = (4, 5)$

En la tercera consulta del primer caso de prueba, se cuentan todas las celdas que pueden encontrarse dentro de al menos una posible configuración del terreno agrícola.

En la cuarta consulta del primer caso de prueba, la única celda del rectángulo considerado puede formar parte del terreno de la granja, por lo que la respuesta es 1.

C. Kerbal Space Program

Tiempo: 2 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

El Centro Espacial Kerbal está preparando una nueva misión de exploración en una nave de última generación. Durante la expedición, Jebediah Kerman planea visitar n planetas. Para el planeta i , a_i representa la cantidad máxima de contenedores de suministros que una misión procedente de Kerbin puede llevar hasta su superficie, mientras que b_i es la cantidad de kerbals que habitan en la colonia del planeta.

Jebediah Kerman quiere transportar provisiones desde Kerbin para las colonias que visitará durante la misión. Las provisiones se almacenan en contenedores, con exactamente x unidades en cada uno. Antes del lanzamiento, la nave será cargada con exactamente $a_1 + \dots + a_n$ contenedores.

Cuando Jebediah aterriza en el planeta i , descarga a_i contenedores y abandona temporalmente la nave. Durante el primer día en el planeta recorre la colonia y conoce a sus habitantes. Desde el segundo día y durante todos los días posteriores, distribuye las provisiones entre los kerbals — cada uno de los b_i habitantes recibe una unidad de provisiones por día. Jebediah abandona el planeta durante la tarde del día en que la cantidad de provisiones restantes es estrictamente menor que la cantidad de habitantes (es decir, tan pronto como ya no sea posible entregar la cantidad correspondiente de provisiones al día siguiente). Las provisiones restantes se dejan almacenadas en la colonia.

Jebediah tiene previsto dedicar exactamente c días a toda la expedición. El tiempo empleado en los vuelos entre los planetas se considera igual a cero. ¿De cuántas maneras puede elegirse el entero positivo x para que la misión planificada dure exactamente c días?

Entrada

La primera línea de la entrada contiene los enteros n y c separados por espacios — la cantidad de planetas que Jebediah Kerman visitará y la cantidad de días que dedicará a la expedición, respectivamente.

Las siguientes n líneas contienen pares de enteros a_i, b_i separados por espacios ($1 \leq i \leq n$) — la cantidad de contenedores que puede llevar al planeta i y la cantidad de habitantes de dicho planeta, respectivamente.

Las restricciones de entrada para obtener 30 puntos son:

- $1 \leq n \leq 100$
- $1 \leq a_i \leq 100$
- $1 \leq b_i \leq 100$
- $1 \leq c \leq 100$

Las restricciones de entrada para obtener 100 puntos son:

- $1 \leq n \leq 10^4$
- $0 \leq a_i \leq 10^9$
- $1 \leq b_i \leq 10^9$
- $1 \leq c \leq 10^9$

Debido a posibles desbordamientos, se recomienda utilizar aritmética de 64 bits. En algunas soluciones incluso la aritmética de 64 bits puede desbordarse. ¡Ten cuidado al realizar los cálculos!

Salida

Imprime un único número k — la cantidad de formas de elegir x para que la expedición dure exactamente c días. Si existen infinitos valores posibles de x , imprime -1.

No utilices el especificador `%lld` para leer o escribir enteros de 64 bits en C++. Es preferible utilizar los flujos `cin`, `cout` o el especificador `%I64d`.

Ejemplo

Entrada

```
2 5
1 5
2 4
```

Salida

```
1
```

Nota

En el primer ejemplo existe un único valor adecuado $x = 5$. Entonces Jebediah Kerman lleva 1 contenedor con 5 unidades de provisiones al primer planeta. Allí permanece durante 2 días: recorre la colonia durante el primer día y entrega cinco unidades de provisiones durante el segundo día. Luego lleva 2 contenedores con 10 unidades de provisiones al segundo planeta. Allí permanece durante 3 días — entrega 4 unidades durante el segundo y el tercer día y deja las 2 unidades restantes almacenadas en la colonia. En total, Jebediah Kerman pasa 5 días en la expedición.

Para $x = 4$ o menos, Jebediah no tendrá suficientes provisiones para el segundo día en el primer planeta, por lo que la expedición terminará demasiado pronto. Para $x = 6$ o más, Jebediah permanecerá al menos un día adicional en el segundo planeta y la expedición durará demasiado.

D. Papers, Please

Tiempo: 1 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

En el puesto fronterizo de Arstotzka llegan cada día cientos de viajeros con documentos, permisos y nombres que deben ser revisados cuidadosamente. Después de varias jornadas de trabajo, el inspector ha aprendido que incluso el detalle más pequeño puede ser suficiente para que un expediente resulte sospechoso.

Recientemente, el Ministerio de Admisión instaló un antiguo sistema automático destinado a clasificar a las personas utilizando únicamente el nombre registrado en sus documentos. Nadie parece recordar quién programó el sistema ni por qué utiliza semejante criterio, pero las órdenes son claras: mientras el equipo continúe funcionando, el inspector debe respetar sus decisiones.

El método utilizado por el sistema es el siguiente: si la cantidad de caracteres distintos presentes en el nombre es impar, el expediente se clasifica bajo el código "IGNORE HIM!". En caso contrario, se clasifica bajo el código "CHAT WITH HER!".

Los mensajes fueron heredados de una versión antigua del software fronterizo y no pueden modificarse, por lo que deben utilizarse exactamente como aparecen, independientemente de su extraño significado.

Se te proporciona el nombre registrado en los documentos de un viajero. Ayuda al inspector de Arstotzka a determinar qué clasificación producirá el sistema.

Entrada

La primera línea contiene una cadena no vacía formada únicamente por letras minúsculas del alfabeto inglés — el nombre registrado. La cadena contiene como máximo 100 letras.

Salida

Si, de acuerdo con el método del sistema, corresponde la primera clasificación, imprime "CHAT WITH HER!" (sin las comillas). En caso contrario, imprime "IGNORE HIM!" (sin las comillas).

Ejemplos

Ejemplo 1

Entrada

```
wjnzbmrr
```

Salida

```
CHAT WITH HER!
```

Ejemplo 2**Entrada**

```
xiaodao
```

Salida

```
IGNORE HIM!
```

Ejemplo 3**Entrada**

```
sevenkplus
```

Salida

```
CHAT WITH HER!
```

Nota

En el primer ejemplo, hay 6 caracteres distintos en "wjmbmr". Estos caracteres son: "w", "j", "m", "z", "b", "r". Por lo tanto, el sistema produce la clasificación "CHAT WITH HER!".

E. Factorio

Tiempo: 2 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

The factory must grow.

Después de varias horas ampliando su fábrica, el ingeniero de Factorio descubrió un nuevo problema en una de sus líneas de producción. Una larga cinta transportadora conecta distintas zonas de procesamiento y mueve materiales entre ensambladoras, hornos y almacenes. A primera vista, todo parecía funcionar correctamente, pero el sistema encargado de coordinar el flujo comenzó a detectar interferencias entre algunos tramos consecutivos.

Para evitar detener toda la fábrica, el ingeniero decidió reconstruir desde cero la distribución de materiales. La nueva línea estará formada por n tramos consecutivos de cinta transportadora. Cada tramo será configurado para manejar una determinada cantidad entera de materiales, y el controlador central analizará la relación existente entre cada par de tramos vecinos. Por razones relacionadas con la sincronización interna de la fábrica, dos conexiones diferentes no pueden producir exactamente el mismo patrón.

El ingeniero debe determinar una configuración a para las cantidades de materiales procesadas a lo largo de la línea, donde cada valor corresponde a uno de sus tramos. El patrón de sincronización entre dos tramos consecutivos está determinado por $\gcd(a_i, a_{i+1})$ *.

La configuración debe elegirse de manera que $\gcd(a_i, a_{i+1})$ sea distinto para todos los $1 \leq i \leq n - 1$. Es decir, ninguna conexión entre tramos consecutivos puede tener el mismo valor de sincronización que otra. Se garantiza que existe al menos una configuración a válida.

El tipo concreto de material transportado no es relevante para el controlador. Lo único importante es la cantidad asignada a cada tramo y que, una vez puesta en funcionamiento la línea, todas las conexiones consecutivas produzcan valores de sincronización diferentes. Una vez encontrada cualquier configuración válida, la cinta podrá incorporarse al resto del sistema de producción.

* $\gcd(x, y)$ se refiere al máximo común divisor de los enteros x e y .

Entrada

Cada prueba contiene múltiples casos de prueba. La primera línea contiene el número de casos de prueba t ($1 \leq t \leq 100$). A continuación se describe cada caso de prueba.

Las siguientes t líneas contienen un entero n ($2 \leq n \leq 10^4$) — la cantidad de tramos consecutivos que tendrá la nueva línea de transporte.

Se garantiza que la suma de n sobre todos los casos de prueba no excede 10^4 .

Salida

Para cada consulta, imprime tu respuesta — una configuración a de n enteros separados por espacios ($1 \leq a_i \leq 10^{18}$).

Ejemplo

Entrada

```
2
3
5
```

Salida

```
1 6 2
134 67 69 207 414
```

Nota

En el primer caso de prueba, la configuración $[1, 6, 2]$ es una respuesta posible. Esto se debe a que $\gcd(1, 6)$ no es igual a $\gcd(6, 2)$.

En términos de la fábrica, las cantidades de materiales elegidas hacen que las conexiones consecutivas de la cinta produzcan patrones diferentes.

En el segundo caso de prueba, la configuración $[134, 67, 69, 207, 414]$ es una respuesta posible. Esto se debe a que los valores de $\gcd(a_i, a_{i+1})$ para todos los i entre 1 y $n - 1$ son distintos. Como referencia, estos valores son 67, 1, 69 y 207.

Por lo tanto, todas las conexiones consecutivas pueden ser diferenciadas correctamente por el controlador de la fábrica.

F. Pokémon

Tiempo: 2 s**Memoria:** 512 MB**Entrada:** entrada estándar**Salida:** salida estándar

Cada año, el Profesor Oak organiza un evento especial para los entrenadores Pokémon de la región. Durante este evento, cada participante debe preparar un Pokémon para entregárselo a otro entrenador. Sin embargo, para que la actividad funcione correctamente, Oak debe organizar cuidadosamente quién será el destinatario de cada entrenador.

n entrenadores numerados desde 1 hasta n participan en el evento. A cada entrenador i se le asigna un entrenador diferente b_i , al cual el entrenador i deberá entregar su Pokémon. Cada entrenador debe ser asignado exactamente a otro entrenador y nadie puede ser asignado a sí mismo (aunque dos entrenadores pueden ser asignados mutuamente). Formalmente, todos los b_i deben ser enteros distintos entre 1 y n , y para cualquier i , debe cumplirse $b_i \neq i$.

Normalmente, el Profesor Oak realiza esta asignación sin consultar las preferencias de los participantes. Este año, sin embargo, ha decidido probar algo diferente. Antes de comenzar el evento, preguntó a todos los entrenadores a quién les gustaría entregar su Pokémon. Cada entrenador i respondió que desea entregárselo al entrenador a_i .

Naturalmente, no siempre será posible satisfacer simultáneamente todas las preferencias. Varios entrenadores pueden haber elegido al mismo destinatario y Oak sigue teniendo que respetar las reglas generales del evento: cada entrenador debe recibir exactamente una asignación y ningún participante puede terminar asignado consigo mismo.

Encuentra una asignación válida b que maximice el número de preferencias de los entrenadores que pueden ser satisfechas.

Entrada

Cada prueba contiene múltiples casos de prueba. La primera línea contiene el número de casos de prueba t ($1 \leq t \leq 10^5$). A continuación se describen los casos de prueba.

Cada caso de prueba consta de dos líneas. La primera línea contiene un único entero n ($2 \leq n \leq 2 \cdot 10^5$) — el número de entrenadores que participan en el evento.

La segunda línea contiene n enteros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n; a_i \neq i$) — las preferencias de los entrenadores en orden desde 1 hasta n .

Se garantiza que la suma de n sobre todos los casos de prueba no excede $2 \cdot 10^5$.

Salida

Para cada caso de prueba, imprime dos líneas.

En la primera línea, imprime un único entero k ($0 \leq k \leq n$) — el número de preferencias satisfechas en tu asignación.

En la segunda línea, imprime n enteros distintos $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq n; b_i \neq i$) — los números de los

entrenadores asignados a los entrenadores $1, 2, \dots, n$.

k debe ser igual al número de valores de i tales que $a_i = b_i$, y debe ser tan grande como sea posible. Si existen múltiples respuestas, imprime cualquiera de ellas.

Ejemplo

Entrada

```
2
3
2 1 2
7
6 4 6 2 4 5 6
```

Salida

```
2
3 1 2
4
6 4 7 2 3 5 1
```

Nota

En el primer caso de prueba, existen dos asignaciones válidas: $[3, 1, 2]$ y $[2, 3, 1]$. La primera asignación satisface dos preferencias, mientras que la segunda satisface solamente una. Por lo tanto, $k = 2$, y la única respuesta correcta es $[3, 1, 2]$.

G. Guitar Hero 3

Tiempo: 1 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

¡La gran presentación está por comenzar! Un jugador de Guitar Hero ha preparado una canción especial y quiere interpretarla frente al público.

La canción contiene n partes. Se necesitan a_i segundos para interpretar la i -ésima parte. El jugador no puede cambiar el orden de las partes de la canción: primero interpreta la parte que tarda a_1 segundos, después — la parte que tarda a_2 segundos, y así sucesivamente. Al terminar su presentación, su puntuación será igual al número de partes que haya logrado interpretar completamente.

Mientras toca, el jugador puede saltarse como máximo una parte de la canción (si se salta más de una parte, el sistema de Guitar Hero detectará demasiados errores y la presentación dejará de considerarse válida).

El escenario estará disponible para la canción durante no más de s segundos. Por ejemplo, si $s = 10$, $a = [100, 9, 1, 1]$, y el jugador se salta la primera parte de la canción, entonces consigue completar dos partes.

Ten en cuenta que también es posible interpretar la canción completa (si hay suficiente tiempo).

Determina qué parte debe saltarse el jugador para obtener el máximo número posible de partes completadas. Si no debería saltarse ninguna parte, imprime 0. Si existen varias respuestas, imprime cualquiera de ellas.

Debes procesar t casos de prueba.

Entrada

La primera línea contiene un entero t ($1 \leq t \leq 100$) — el número de casos de prueba.

La primera línea de cada caso de prueba contiene dos enteros n y s ($1 \leq n \leq 10^5, 1 \leq s \leq 10^9$) — el número de partes de la canción y el máximo número de segundos durante los cuales estará disponible el escenario, respectivamente.

La segunda línea de cada caso de prueba contiene n enteros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — el tiempo necesario para interpretar cada parte de la canción.

Se garantiza que la suma de n sobre todos los casos de prueba no excede 10^5 .

Salida

Para cada caso de prueba imprime un entero — el número de la parte que el jugador debe saltarse para obtener el máximo número de partes completadas. Si no debe saltarse ninguna parte, imprime 0.

Ejemplo

Entrada

```
3
7 11
2 9 1 3 18 1 4
4 35
11 9 10 7
1 8
5
```

Salida

```
2
1
0
```

Nota

En el primer caso de prueba, si el jugador se salta la segunda parte, consigue completar tres partes de la canción.

En el segundo caso de prueba, no importa qué parte de la canción se salte el jugador.

En el tercer caso de prueba, el jugador puede interpretar la canción completa.

H. Moving Out

Tiempo: 1 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

Los F.A.R.T.s de Smooth Moves tienen que completar una nueva mudanza. Después de cargar el camión, descubren que hay una gran pila de n objetos, numerados desde 1 hasta n ; el objeto que se encuentra en la parte superior tiene número a_1 , el siguiente objeto es a_2 , y así sucesivamente; el objeto situado en el fondo tiene número a_n . Todos los números son distintos.

Los F.A.R.T.s tienen una lista de m objetos **distintos** que deben descargar: $b_1, b_2, \dots, b_m$. Deben descargarlos **en el orden en que aparecen en la lista**.

Para descargar un objeto, primero tienen que encontrarlo dentro de la pila retirando todos los objetos que se encuentran sobre él, sacar el objeto correspondiente y después devolver todos los objetos retirados a la parte superior de la pila. Por lo tanto, si hay k objetos encima del objeto que quieren descargar, realizar todo el proceso requiere $2k + 1$ segundos. Afortunadamente, los empleados de Smooth Moves pueden acelerar el trabajo: cuando vuelven a colocar los objetos retirados en la pila, pueden reordenarlos como deseen (únicamente aquellos que estaban por encima del objeto que querían retirar; los objetos que se encontraban debajo no pueden verse afectados de ninguna manera).

Determina el tiempo mínimo necesario para descargar todos los objetos, suponiendo que los F.A.R.T.s conocen de antemano la lista completa de objetos que deberán retirar y reorganizan la pila de manera óptima. No pueden cambiar el orden de los objetos de la lista ni interactuar con la pila de ninguna otra manera.

Tu programa debe responder t casos de prueba diferentes.

Entrada

La primera línea contiene un entero t ($1 \leq t \leq 100$) — el número de casos de prueba.

A continuación siguen los casos de prueba, cada uno representado por tres líneas.

La primera línea contiene dos enteros n y m ($1 \leq m \leq n \leq 10^5$) — el número de objetos en la pila y el número de objetos que los F.A.R.T.s deben descargar, respectivamente.

La segunda línea contiene n enteros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$, todos los a_i son únicos) — el orden de los objetos dentro de la pila.

La tercera línea contiene m enteros $b_1, b_2, \dots, b_m$ ($1 \leq b_i \leq n$, todos los b_i son únicos) — la lista ordenada de objetos que deben descargar.

La suma de n sobre todos los casos de prueba no excede 10^5 .

Salida

Para cada caso de prueba, imprime un entero — el número mínimo de segundos que los F.A.R.T.s necesitan para descargar todos los objetos, suponiendo que reordenan los objetos de manera óptima cada vez que los devuelven a la pila.

Ejemplo

Entrada

```
2
3 3
3 1 2
3 2 1
7 2
2 1 7 3 4 5 6
3 1
```

Salida

```
5
8
```


I. Balatro

Tiempo: 2 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

Una partida particularmente extraña de Balatro está a punto de comenzar. Entre Jokers, multiplicadores y reglas que parecen cambiar justo cuando uno cree haber entendido la partida, ha aparecido un nuevo modificador que presta especial atención a la posición y al valor de las cartas.

Sobre la mesa se encuentra una fila representada por un arreglo $a_1, a_2, \dots, a_n$ que consiste de n enteros **distintos**. Cada número representa el valor asignado a la carta que ocupa esa posición. Como todos los valores son diferentes, ninguna de las cartas comparte exactamente el mismo valor con otra.

El nuevo Joker no está interesado en palos, colores ni manos tradicionales. En su lugar, observa simultáneamente dos cartas y decide si forman una pareja que considera especialmente agradable. Curiosamente, para realizar esta comprobación no basta con mirar solamente los valores de las cartas: también importa la posición exacta que cada una ocupa en la fila.

Además, el orden importa para identificar cada pareja. La primera carta considerada debe aparecer antes que la segunda, por lo que solamente se tienen en cuenta pares de posiciones en el orden en que aparecen sobre la mesa.

Según la peculiar regla impuesta por el Joker, dos cartas forman una pareja válida únicamente cuando sus valores y sus posiciones satisfacen exactamente la condición indicada a continuación.

Cuenta el número de pares de índices (i, j) tales que $i < j$ y $a_i \cdot a_j = i + j$.

Entrada

La primera línea contiene un entero t ($1 \leq t \leq 10^4$) — el número de casos de prueba. Luego siguen t casos.

La primera línea de cada caso de prueba contiene un entero n ($2 \leq n \leq 10^5$) — la longitud del arreglo a .

La segunda línea de cada caso de prueba contiene n enteros separados por espacios $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 2 \cdot n$) — el arreglo a . Se garantiza que todos sus elementos son **distintos**.

Se garantiza que la suma de n sobre todos los casos de prueba no excede $2 \cdot 10^5$.

Salida

Para cada caso de prueba, imprime el número de pares de índices (i, j) tales que $i < j$ y $a_i \cdot a_j = i + j$.

Ejemplo

Entrada

```
3
2
3 1
3
6 1 5
5
3 1 5 9 2
```

Salida

```
1
1
3
```

Nota

Para el primer caso de prueba, el único par que satisface las restricciones es $(1, 2)$, ya que $a_1 \cdot a_2 = 1 + 2 = 3$.

Para el segundo caso de prueba, el único par que satisface las restricciones es $(2, 3)$.

Para el tercer caso de prueba, los pares que satisfacen las restricciones son $(1, 2)$, $(1, 5)$, y $(2, 3)$.

J. Minecraft

Tiempo: 2 s**Memoria:** 512 MB**Entrada:** entrada estándar**Salida:** salida estándar

Después de explorar durante demasiado tiempo las minas y transportar manualmente recursos entre distintos puntos de su mundo, Steve decidió construir un sistema ferroviario subterráneo mucho más grande de lo estrictamente necesario. La red conecta minas, almacenes, granjas automáticas y otras instalaciones mediante estaciones de vagonetas distribuidas por todo el subsuelo.

La construcción de los túneles y los rieles ya está terminada. Sin embargo, Steve todavía debe configurar los mecanismos de Redstone y los rieles impulsores que controlan el sentido de circulación de cada conexión. Cada tramo podrá utilizarse en un único sentido una vez que el sistema quede configurado.

Se te proporciona una red no dirigida con n estaciones y m tramos de riel. Las estaciones están numeradas desde 1 hasta n . La red no contiene conexiones de una estación consigo misma ni múltiples tramos entre el mismo par de estaciones.

Tu tarea consiste en convertir la red ferroviaria en una red dirigida eligiendo una dirección para cada tramo. Una vez configurados los rieles, llamaremos *recorrido alternante* a una secuencia de estaciones $v_1, v_2, \dots, v_k$, donde k puede ser arbitrariamente grande y cualquier estación puede repetirse cualquier número de veces, si:

- el tramo (v_1, v_2) está dirigido desde v_1 hacia v_2 ;
- el tramo (v_2, v_3) está dirigido desde v_3 hacia v_2 ;
- el tramo (v_3, v_4) está dirigido desde v_3 hacia v_4 ;
- el tramo (v_4, v_5) está dirigido desde v_5 hacia v_4 ;
- y así sucesivamente.

Esta alternancia representa el comportamiento que Steve espera obtener de los mecanismos de impulso instalados entre estaciones consecutivas. Aunque una vagoneta puede atravesar una misma zona varias veces y un recorrido no tiene por qué ser simple, el patrón de orientación de los tramos debe respetarse durante todo el trayecto.

Steve llama *estable* a una estación v si todos los recorridos (no necesariamente simples) de la red original que comienzan en la estación v son *alternantes* en la red dirigida resultante.

Antes de activar simultáneamente todas las líneas de vagonetas, Steve quiere configurar los mecanismos de cada tramo de manera que la mayor cantidad posible de estaciones cumpla esta condición.

¿Cuál es el máximo número de estaciones que pueden hacerse *estables* después de dirigir los tramos?

Entrada

Cada prueba contiene múltiples casos de prueba. La primera línea contiene el número de casos de prueba t ($1 \leq t \leq 10^4$). A continuación se describe cada caso de prueba.

La primera línea contiene dos enteros n y m ($1 \leq n \leq 2 \cdot 10^5$; $0 \leq m \leq 2 \cdot 10^5$) — el número de estaciones y tramos de riel de la red, respectivamente.

Cada una de las siguientes m líneas contiene dos enteros v y u ($1 \leq v, u \leq n$) — la descripción de los tramos

de riel de la red.

Restricciones adicionales sobre la entrada:

- La red dada no contiene conexiones de una estación consigo misma ni múltiples tramos entre el mismo par de estaciones;
- La suma de n sobre todos los casos de prueba no excede $2 \cdot 10^5$;
- La suma de m sobre todos los casos de prueba no excede $2 \cdot 10^5$.

Salida

Para cada caso de prueba, imprime un único entero — el máximo número de estaciones que pueden hacerse *estables* después de dirigir los tramos.

Ejemplo

Entrada

```
4
8 9
1 3
1 4
2 3
2 4
5 6
6 7
7 8
8 5
6 8
4 0
6 2
1 5
2 3
1 0
```

Salida

```
2
4
4
1
```

K. FIFA

Tiempo: 2 s**Memoria:** 512 MB**Entrada:** entrada estándar**Salida:** salida estándar

Durante una sesión de entrenamiento de FIFA, n jugadores se colocan en una fila, numerados desde 1 hasta n de izquierda a derecha.

El entrenador ha preparado un ejercicio de pases en el que cada jugador ya sabe qué hará cuando reciba el balón. Para cada jugador, se conoce si pasará el balón al compañero que se encuentra inmediatamente a su izquierda o al compañero que se encuentra inmediatamente a su derecha. Esta información está representada por una cadena s de n caracteres. Cada carácter de la cadena es L o R, donde s_i es L si el i -ésimo jugador pasa el balón al jugador $(i - 1)$, o s_i es R si el i -ésimo jugador pasa el balón al jugador $(i + 1)$.

Como los jugadores de los extremos solo tienen un compañero disponible dentro de la formación, el primer jugador siempre pasa el balón al segundo y el último siempre se lo pasa al penúltimo (en otras palabras, la cadena s comienza con el carácter R y termina con el carácter L).

Antes de comenzar el entrenamiento, el cuerpo técnico quiere observar cómo se propagará el balón por la formación. Cada jugador seguirá estrictamente la dirección de pase que tiene asignada, incluso si vuelve a recibir el balón varias veces durante el ejercicio.

Considera el siguiente proceso:

- primero, el primer jugador recibe el balón;
- luego, exactamente n veces, ocurre lo siguiente: el jugador que actualmente tiene el balón se lo pasa a su compañero vecino (de acuerdo con las reglas descritas anteriormente).

Tu tarea es determinar cuántos jugadores recibirán el balón al menos una vez durante este proceso.

Entrada

La primera línea contiene un único entero t ($1 \leq t \leq 10000$) — el número de casos de prueba.

Cada caso de prueba consta de dos líneas:

- la primera línea contiene un único entero n ($2 \leq n \leq 50$) — el número de jugadores;
- la segunda línea contiene s — una secuencia de n caracteres L y R. El primer carácter de la secuencia es R, y el último es L.

Salida

Para cada caso de prueba, imprime un entero — el número de jugadores que recibirán el balón al menos una vez durante el proceso descrito.

Ejemplo

Entrada

```
3
4
RLRL
6
RRRRRL
9
RRLRRRRRL
```

Salida

```
2
6
3
```

Nota

En el primer ejemplo, el jugador 1 recibe el balón y se lo pasa al jugador 2, quien lo devuelve al jugador 1, quien vuelve a pasárselo al jugador 2, y así sucesivamente. Solo los jugadores 1 y 2 reciben el balón.

En el segundo ejemplo, el jugador 1 pasa el balón al jugador 2, quien se lo pasa al jugador 3, quien se lo pasa al jugador 4, quien se lo pasa al jugador 5, quien se lo pasa al jugador 6, quien lo devuelve al jugador 5. Todos los jugadores reciben el balón al menos una vez.

L. Terraria

Tiempo: 1 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

Durante una expedición en Terraria, el jugador decidió aprovechar una gran cantidad de bloques que llevaba en su inventario para construir varias torres. En total, construyó n torres, donde la i -ésima torre tiene una altura de h_i bloques.

Antes de continuar con la construcción, quiere que todas las torres terminen teniendo exactamente la misma altura. Para conseguirlo, debe elegir un entero x_i para cada torre i y aumentar su altura en x_i bloques **exactamente una vez**.

Por ejemplo, si $h = [1, 3, 2, 2]$, $x = [3, 2, 2, 8]$, entonces después de añadir los bloques las alturas serán $[4, 5, 4, 10]$.

Sin embargo, para hacer la construcción un poco más interesante, el jugador decide imponer una restricción adicional. Elegirá un entero k y exigirá que cada cantidad x_i utilizada en las torres satisfaga $1 \leq x_i \leq k$.

Su objetivo sigue siendo conseguir que, después de realizar exactamente una ampliación en cada torre, todas tengan la misma altura.

Ayuda al jugador a encontrar el menor valor posible de k para el cual sea posible completar la construcción.

Entrada

La primera línea contiene un único entero t ($1 \leq t \leq 10^4$) — el número de casos de prueba.

Luego siguen t casos de prueba.

La primera línea de cada caso de prueba contiene un único entero n ($1 \leq n \leq 5$).

La segunda línea contiene n enteros $h_1, h_2, \dots, h_n$ ($1 \leq h_i \leq 6$).

Salida

Para cada caso de prueba, imprime un único entero — el valor mínimo de k tal que sea posible hacer que todas las torres tengan la misma altura.

Ejemplo

Entrada

```
4
2
1 3
3
2 6 4
5
5 4 6 6 1
4
3 3 3 3
```

Salida

```
3
5
6
1
```


M. Oxygen Not Included

Tiempo: 1 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

En una de las nuevas colonias de Oxygen Not Included, los Duplicantes han terminado de instalar un complejo sistema de control formado por una matriz rectangular de módulos. El sistema se utiliza para coordinar distintas partes de la colonia, desde bombas y conductos hasta mecanismos de automatización.

La instalación consta de una matriz a formada por n filas y m columnas. La celda situada en la i -ésima fila desde arriba y en la j -ésima columna desde la izquierda contiene un valor $a_{i,j}$ que representa el nivel de calibración de ese módulo.

Después de activar el sistema, los Duplicantes descubrieron un problema bastante serio. Si dos módulos adyacentes contienen exactamente el mismo valor, sus señales interfieren entre sí y el sistema deja de funcionar correctamente. Una matriz se considera **buena** si ningún par de celdas adyacentes contiene el mismo valor, donde dos celdas se consideran adyacentes si comparten un lado.

Afortunadamente, el sistema de automatización permite realizar una pequeña recalibración antes de poner definitivamente en funcionamiento la colonia. Para cada módulo, es posible **incrementar su valor en uno**. No es posible disminuirlo ni modificarlo de ninguna otra manera.

Tu tarea consiste en ayudar a los Duplicantes a transformar la configuración actual en una que pueda funcionar sin interferencias.

Más formalmente, encuentra una matriz buena b que satisfaga la siguiente condición —

- Para todos los (i, j) válidos, se debe cumplir que $b_{i,j} = a_{i,j}$ o $b_{i,j} = a_{i,j} + 1$.

Para las restricciones de este problema, puede demostrarse que siempre existe una matriz b que cumple las condiciones. Si existen varias configuraciones posibles, puedes imprimir cualquiera de ellas. Ten en cuenta que no es necesario minimizar el número de incrementos realizados.

Entrada

Cada prueba contiene múltiples casos de prueba. La primera línea contiene el número de casos de prueba t ($1 \leq t \leq 10$). A continuación se describen los casos de prueba.

La primera línea de cada caso de prueba contiene dos enteros n, m ($1 \leq n \leq 100, 1 \leq m \leq 100$) — el número de filas y columnas, respectivamente.

Las siguientes n líneas contienen cada una m enteros. El j -ésimo entero de la i -ésima línea es $a_{i,j}$ ($1 \leq a_{i,j} \leq 10^9$).

Salida

Para cada caso, imprime n líneas, cada una con m enteros. El j -ésimo entero de la i -ésima línea debe ser $b_{i,j}$.

Ejemplo

Entrada

```
3
3 2
1 2
4 5
7 8
2 2
1 1
3 3
2 2
1 3
2 2
```

Salida

```
1 2
5 6
7 8
2 1
4 3
2 4
3 2
```

Nota

En todos los casos, puedes verificar que ningún par de celdas adyacentes contiene el mismo valor y que b es igual a a con algunos valores incrementados en uno.

N. StarCraft

Tiempo: 4 s

Memoria: 512 MB

Entrada: entrada estándar

Salida: salida estándar

El Sector Koprulu tiene n bases, numeradas con enteros desde 1 hasta n . En la base i se encuentran b_i unidades Terran y w_i unidades Zerg.

Algunas bases están conectadas mediante rutas interestelares. Diremos que dos bases son adyacentes si existe una ruta entre ellas. Un camino simple desde la base x hasta la base y está dado por una secuencia $s_0, \dots, s_p$ de bases distintas, donde p es un entero no negativo, $s_0 = x$, $s_p = y$, y s_{i-1} y s_i son adyacentes para cada $i = 1, \dots, p$. Las rutas del Sector Koprulu están organizadas de tal manera que para cualesquiera dos bases x e y , existe un único camino simple desde x hasta y . Debido a esto, estrategias y científicos están de acuerdo en que la red es, de hecho, un árbol.

Un sector es un conjunto no vacío V de bases tal que para cualesquiera dos x e y pertenecientes a V , existe un camino simple desde x hasta y cuyas bases intermedias pertenecen todas a V .

Un conjunto de sectores $\mathcal{P}$ se denomina una partición si cada una de las n bases está contenida exactamente en uno de los sectores de $\mathcal{P}$. En otras palabras, ningún par de sectores de $\mathcal{P}$ comparte una base en común y, en conjunto, contienen las n bases.

Terran y Zerg disputan el dominio del Sector Koprulu. Supongamos que $\mathcal{P}$ es una partición de las bases en m sectores $V_1, \dots, V_m$. En cada sector se produce un enfrentamiento local entre ambas fuerzas. Si existen **estrictamente** más unidades Zerg que unidades Terran en un sector, se dice que los Zerg *ganan* ese sector. En caso contrario, los Terran *ganan*. La facción que gane en la mayor cantidad de sectores tendrá el control de la campaña.

Como es de esperar, las unidades Terran siempre luchan por los Terran y las unidades Zerg siempre luchan por los Zerg. Ninguna unidad permanece neutral durante la batalla por el Sector Koprulu.

Sarah Kerrigan, con la ayuda de Abathur, quiere que los Zerg ganen. Kerrigan tiene el poder de elegir cómo dividir la red en exactamente m sectores. Si elige la partición de manera óptima, determina el máximo número de sectores en los que los Zerg pueden ganar.

Entrada

La primera línea de la entrada contiene un único entero t ($1 \leq t \leq 100$), que representa el número de casos de prueba. Las siguientes líneas contienen las descripciones de los casos de prueba.

La primera línea de cada caso de prueba contiene dos enteros separados por espacios n y m ($1 \leq m \leq n \leq 3000$). La segunda línea contiene n enteros separados por espacios $b_1, b_2, \dots, b_n$ ($0 \leq b_i \leq 10^9$). La tercera línea contiene n enteros separados por espacios $w_1, w_2, \dots, w_n$ ($0 \leq w_i \leq 10^9$). Las siguientes $n - 1$ líneas describen los pares de bases adyacentes. En particular, la i -ésima de ellas contiene dos enteros separados por espacios x_i e y_i ($1 \leq x_i, y_i \leq n$, $x_i \neq y_i$), que representan los números de dos bases adyacentes. Se garantiza que estos pares forman un árbol.

Se garantiza que la suma de n en un único archivo es como máximo 10^5 .

Salida

Para cada caso de prueba, imprime una única línea que contenga un único entero que represente el máximo número de sectores en los que los Zerg ganan, entre todas las particiones de la red en m sectores.

Ejemplo

Entrada

```
2
4 3
10 160 70 50
70 111 111 0
1 2
2 3
3 4
2 1
143 420
214 349
2 1
```

Salida

```
2
0
```

Nota

En el primer caso de prueba, debemos dividir las $n = 4$ bases en $m = 3$ sectores. Podemos hacer que los Zerg ganen en 2 sectores mediante la siguiente partición: $\{\{1, 2\}, \{3\}, \{4\}\}$. En esta partición,

- los Zerg ganan en el sector $\{1, 2\}$, con 181 unidades frente a las 170 unidades Terran;
- los Zerg también ganan en el sector $\{3\}$, con 111 unidades frente a las 70 unidades Terran;
- los Zerg pierden en el sector $\{4\}$, con 0 unidades frente a las 50 unidades Terran.

Por lo tanto, los Zerg ganan en 2 sectores, y puede demostrarse que esta es la máxima cantidad posible.

En el segundo caso de prueba, debemos dividir las $n = 2$ bases en $m = 1$ sector. Solo existe una manera de hacerlo: $\{\{1, 2\}\}$. En el único sector de esta partición, los Zerg tienen 563 unidades y los Terran también tienen 563 unidades. Los Zerg necesitan estrictamente más unidades para ganar. Por lo tanto, los Zerg no ganan ningún sector.

O. Among Us

Tiempo: 2 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

Durante una tranquila ronda en The Skeld, la tripulación descubre que alguien ha saboteado nuevamente el sistema eléctrico. Las luces comienzan a fallar, varias alarmas se activan y, como era de esperar, nadie parece haber visto quién entró o salió de Electrical.

Uno de los tripulantes consigue llegar al panel principal antes de que la situación empeore. El sistema está formado por una secuencia de interruptores que solo pueden encontrarse en dos estados posibles. Debido al sabotaje, algunos de ellos se encuentran activados cuando deberían estar desactivados.

Como recompensa por reparar completamente el sistema, la tripulación ha prometido al responsable nada menos que 1 000 000 créditos. Sin embargo, para conseguirlos deberá restaurar correctamente todo el panel.

El estado actual del panel se representa mediante una cadena binaria s de longitud n y un entero k .

El cableado del panel ha sido modificado de una manera bastante peculiar. El tripulante puede realizar una cantidad ilimitada de movimientos. En un movimiento, puede elegir un entero i ($1 \leq i \leq n - k$) e invertir los caracteres en las posiciones i y $i + k$, es decir, cambiar 0 por 1 y 1 por 0.

Esto ocurre porque ambos interruptores están conectados al mismo circuito: al modificar el estado de uno de estos pares, los dos cambian simultáneamente. El tripulante no puede modificar individualmente ninguno de ellos ni realizar ninguna otra operación sobre el panel.

Por ejemplo, si $s = 10110$ y $k = 2$, entonces, al elegir $i = 2$, el tripulante invierte los caracteres en las posiciones 2 y 4:

[10110 $\rightarrow$ 11100]

Para reparar completamente el sabotaje, todos los interruptores deben quedar desactivados. En otras palabras, el tripulante necesita conseguir que toda la cadena s sea igual a cero.

Si logra hacerlo, Electrical volverá a funcionar, obtendrá su recompensa y la tripulación podrá continuar con sus tareas. De lo contrario, probablemente habrá problemas bastante más importantes que descubrir quién es el impostor.

Determina si es posible dejar todo el panel en estado cero utilizando las operaciones permitidas.

Entrada

La primera línea contiene un único entero t ($1 \leq t \leq 10^4$) — el número de casos de prueba.

La primera línea de cada caso de prueba contiene dos enteros n y k ($1 \leq k \leq n \leq 2 \cdot 10^5$).

La segunda línea de cada caso de prueba contiene una cadena binaria s de longitud n .

Se garantiza que la suma de n sobre todos los casos de prueba no excede $2 \cdot 10^5$.

Salida

Para cada caso de prueba, imprime "YES" si el tripulante puede hacer que toda la cadena sea igual a cero, y "NO" en caso contrario.

Puedes imprimir "YES", "NO" utilizando cualquier combinación de mayúsculas y minúsculas (por ejemplo, "yES", "yes", y "Yes" también serán aceptadas).

Ejemplo

Entrada

```
5
4 2
1010
3 2
111
3 3
111
3 1
110
1 1
1
```

Salida

```
YES
NO
NO
YES
NO
```

P. Portal 2

Tiempo: 2 s**Memoria:** 256 MB**Entrada:** entrada estándar**Salida:** salida estándar

Esta es la versión fácil del problema. En esta versión, $m = 1$.

GLaDOS está preparando una nueva cámara de pruebas en Aperture Science. Para ello dispone de dos arreglos de valores a y b , que contienen n y m elementos respectivamente. Los valores del arreglo a representan la configuración actual de los módulos de la cámara, mientras que los valores de b corresponden a parámetros disponibles en el sistema de calibración.

Para **cada** entero i desde 1 hasta n , GLaDOS puede realizar la siguiente operación **como máximo una vez**:

- Elegir un entero j tal que $1 \leq j \leq m$. Establecer $a_i := b_j - a_i$. Ten en cuenta que a_i puede volverse no positivo como resultado de esta operación.

Antes de permitir que un sujeto de pruebas entre en la cámara, GLaDOS necesita determinar si es posible dejar a ordenado de forma no decreciente* realizando la operación anterior cierta cantidad de veces.

* a está ordenado de forma no decreciente si $a_1 \leq a_2 \leq \dots \leq a_n$.

Entrada

La primera línea contiene un entero t ($1 \leq t \leq 10^4$) — el número de casos de prueba.

La primera línea de cada caso de prueba contiene dos enteros n y m ($1 \leq n \leq 2 \cdot 10^5$, $m = 1$).

La siguiente línea de cada caso de prueba contiene n enteros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

La siguiente línea de cada caso de prueba contiene m enteros $b_1, b_2, \dots, b_m$ ($1 \leq b_i \leq 10^9$).

Se garantiza que la suma de n y la suma de m sobre todos los casos de prueba no excede $2 \cdot 10^5$.

Salida

Para cada caso de prueba, si es posible ordenar a de forma no decreciente, imprime "YES".^{en} una nueva línea. En caso contrario, imprime "NO".^{en} una nueva línea.

Puedes imprimir la respuesta utilizando cualquier combinación de mayúsculas y minúsculas. Por ejemplo, las cadenas "yEs", "yes", y "Yes" también serán reconocidas como respuestas positivas.

Ejemplo

Entrada

```
5
1 1
5
9
3 1
1 4 3
3
4 1
1 4 2 5
6
4 1
5 4 10 5
4
3 1
9 8 7
8
```

Salida

```
YES
NO
YES
NO
YES
```

Nota

En el primer caso de prueba, $[5]$ ya está ordenado.

En el segundo caso de prueba, puede demostrarse que es imposible.

En el tercer caso de prueba, podemos establecer $a_3 := b_1 - a_3 = 6 - 2 = 4$. La secuencia $[1, 4, 4, 5]$ queda ordenada de forma no decreciente.

En el último caso de prueba, podemos aplicar operaciones sobre cada índice. La secuencia se convierte en $[-1, 0, 1]$, que está ordenada de forma no decreciente.